

GRAVEATLAS

Phase 3 — Contributions, Authentication, Moderation & Data Quality

Full copy-paste implementation prompt for the Base44 agent.

COPY-PASTE MASTER PROMPT

GRAVEATLAS – PHASE 3
CONTRIBUTIONS, AUTHENTICATION, MODERATION & DATA QUALITY

Phase 1 and Phase 2 must already be complete.

Continue from the existing GraveAtlas implementation.

IMPORTANT:

- Do NOT rebuild Phase 1 or Phase 2.
- Reuse the existing Android app, Cloudflare Worker/API, GitHub App, public data repository, schemas, search, map and public data.
- Do NOT start Phase 4.
- Do NOT add paid services without explicit approval.
- Do NOT fabricate cemetery, grave, person or historical information.
- Do NOT allow users to directly write to the public GitHub dataset.
- Preserve provenance and audit history.
- Do NOT expose private contributor, moderator or administrator information.
- Complete only Phase 3, then stop.

=====

PART 1 – AUTHENTICATION FOUNDATION

=====

Review and implement the authentication foundation required for contributor functionality.

Support the project's existing authentication mechanism.

Verify:

- sign-in
- sign-out
- session handling
- session expiration
- unauthorized state
- authenticated state

Do not store authentication secrets insecurely.

=====

PART 2 – USER IDENTITY

=====

Define the minimum user identity required for contributions.

Avoid collecting unnecessary personal information.

Use stable internal identifiers where appropriate.

Do not expose internal user IDs publicly unless required.

=====

PART 3 – AUTHORIZATION

=====

Define roles and permissions.

At minimum distinguish where applicable:

PUBLIC USER
AUTHENTICATED CONTRIBUTOR
MODERATOR
ADMINISTRATOR

Apply server-side authorization.

Never rely only on Android UI restrictions.

=====

PART 4 – CONTRIBUTION CREATION

=====

Implement the contribution workflow.

A user should be able to create a contribution containing appropriate information such as:

- cemetery
- record/person information
- location
- source
- notes
- correction information
- media reference where supported

Validate all fields.

Do not publish immediately.

=====

PART 5 – CONTRIBUTION DRAFTS

=====

Support drafts where appropriate.

Drafts should be distinguishable from submitted contributions.

A draft must not appear as authoritative public information.

=====

PART 6 – CONTRIBUTION VALIDATION

=====

Validate:

- required fields
- data types
- identifiers
- dates
- coordinates
- text length
- source information
- media references

Reject malformed submissions safely.

=====

PART 7 – DUPLICATE DETECTION

=====

Detect likely duplicate contributions where practical.

Compare appropriate identifiers and public information.

Do not automatically delete legitimate historical records because they look similar.

Flag possible duplicates for review.

=====

PART 8 – CONTRIBUTION STATUS

=====

Define clear statuses such as:

DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
NEEDS_CORRECTION
PUBLISHED
FAILED

Use only statuses actually supported by the workflow.

Users must be able to understand the status of their own submissions.

=====

PART 9 – CONTRIBUTION HISTORY

=====

Allow contributors to view their own contribution history where appropriate.

Do not expose:

- moderator notes
- private administrator information
- internal security data

unless explicitly authorized.

=====

PART 10 – MODERATION QUEUE

=====

Create the moderation queue foundation.

Moderators should be able to see:

- pending contributions
- source information
- relevant public record context
- submission status
- contributor information only when necessary

=====

PART 11 – MODERATION ACTIONS

=====

Support appropriate moderation actions:

- approve
- reject
- request correction
- flag
- merge/duplicate review where supported

Every important moderation action should be auditable.

=====

PART 12 – MODERATION NOTES

=====

Allow internal moderation notes where appropriate.

Moderation notes must never become public data accidentally.

Do not expose them through public API routes.

=====

PART 13 – SOURCE / PROVENANCE

=====

Every approved contribution should retain provenance.

Track appropriate information such as:

- source
- contributor
- submission
- review event
- publication event
- timestamp

Do not fabricate provenance.

=====

PART 14 – CORRECTION WORKFLOW

=====

Create the correction workflow.

Users can report potentially incorrect public information.

A correction should:

1. identify the affected record
2. describe the issue
3. provide supporting information
4. enter moderation
5. be reviewed

6. produce an auditable result

Do not directly overwrite authoritative data from the client.

=====
PART 15 – DATA QUALITY FLAGS
=====

Create quality flags for issues such as:

- duplicate
- missing source
- invalid coordinates
- conflicting dates
- incomplete record
- possible identity conflict
- suspicious submission

Flags should support review rather than automatically deleting data.

=====
PART 16 – ABUSE PROTECTION
=====

Protect contribution endpoints against abuse.

Use appropriate:

- authentication requirements
- rate limits
- request limits
- input validation
- duplicate protection

Do not make legitimate contributions unnecessarily difficult.

=====
PART 17 – MEDIA / EVIDENCE FOUNDATION
=====

If media/evidence uploads are supported:

- validate file type
- validate size
- prevent executable uploads
- protect private material
- associate media with the correct contribution
- prevent unauthorized access

If media uploads are not yet implemented:

create the data/API foundation without pretending upload support exists.

=====
PART 18 – ADMIN / MODERATOR SECURITY
=====

Verify privileged operations require server-side authorization.

Check:

- role validation
- session validation
- operation authorization
- audit logging

Do not trust role information supplied directly by the client.

=====
PART 19 – AUDIT LOGGING
=====

Record important events:

- contribution created
- contribution submitted
- moderation decision
- correction submitted
- correction decision
- publication request

- publication result
- privileged operation

Do not log secrets.

=====

PART 20 – PUBLICATION BOUNDARY

=====

Approved data must pass through a controlled publication process.

Flow:

Approved
→ validated
→ publication preparation
→ GitHub App
→ public repository
→ validation
→ published

A normal user must never receive repository write credentials.

=====

PART 21 – PUBLICATION FAILURE

=====

If publication fails:

- retain approved contribution state
- record failure
- allow safe retry
- prevent duplicate publication
- do not mark it published prematurely

=====

PART 22 – MODERATION UI

=====

Create the moderator interface.

Include:

- queue
- contribution details
- source/provenance
- approve
- reject
- correction request
- notes
- status

Do not expose admin-only information to ordinary contributors.

=====

PART 23 – CONTRIBUTOR UI

=====

Create contributor screens for:

- new contribution
- drafts
- submitted contributions
- status
- correction requests
- contribution history

Keep the workflow understandable.

=====

PART 24 – ERROR / OFFLINE HANDLING

=====

Contributions must handle:

- no network
- timeout
- validation error
- authentication expiration
- server failure

- retry

Never tell a user that a contribution was submitted unless the server confirmed it.

Prevent duplicate submissions after retry.

=====
PART 25 – SECURITY TESTING
=====

Test:

- unauthorized contribution
- unauthorized moderation
- unauthorized publication
- role escalation attempts
- invalid contribution
- oversized input
- duplicate submission
- replayed submission
- expired authentication
- public access to private moderation data

All tests must be safe.

=====
PART 26 – DATA QUALITY TESTING
=====

Test:

- duplicate records
- invalid dates
- invalid coordinates
- missing sources
- conflicting information
- correction workflow
- moderation workflow
- provenance preservation

Do not fabricate historical data.

=====
PART 27 – DOCUMENTATION
=====

Create/update:

docs/AUTHENTICATION.md
docs/AUTHORIZATION.md
docs/CONTRIBUTIONS.md
docs/MODERATION.md
docs/CORRECTIONS.md
docs/PROVENANCE.md
docs/DATA-QUALITY.md
docs/PUBLICATION-WORKFLOW.md

Documentation must describe actual implementation.

=====
PHASE 3 ACCEPTANCE GATE
=====

Verify:

- [] Authentication
- [] User identity
- [] Authorization
- [] Contribution creation
- [] Drafts
- [] Contribution validation
- [] Duplicate detection
- [] Contribution statuses
- [] Contribution history
- [] Moderation queue
- [] Moderation actions
- [] Private moderation notes
- [] Provenance
- [] Correction workflow
- [] Data-quality flags

[] Abuse protection
[] Media/evidence foundation
[] Moderator/admin security
[] Audit logging
[] Publication boundary
[] Publication failure handling
[] Moderator UI
[] Contributor UI
[] Offline/error handling
[] Security tests
[] Data-quality tests
[] Documentation

=====

FINAL REPORT

=====

Return:

PHASE 3 STATUS:
READY / NOT READY

AUTHENTICATION:
PASS / FAIL

AUTHORIZATION:
PASS / FAIL

CONTRIBUTIONS:
PASS / FAIL

MODERATION:
PASS / FAIL

CORRECTIONS:
PASS / FAIL

PROVENANCE:
PASS / FAIL

DATA QUALITY:
PASS / FAIL

ABUSE PROTECTION:
PASS / FAIL

PUBLICATION:
PASS / FAIL

AUDIT:
PASS / FAIL

SECURITY:
PASS / FAIL

TESTS:
PASS / FAIL

DOCUMENTATION:
PASS / FAIL

=====

ACTUAL TEST RESULTS

=====

Report actual results only.

Authentication:
[actual result]

Authorization:
[actual result]

Contribution:
[actual result]

Moderation:
[actual result]

Correction:

[actual result]

Duplicate protection:
[actual result]

Publication:
[actual result]

Security:
[actual result]

Data quality:
[actual result]

Do NOT invent results.

=====
BLOCKERS
=====

List every issue preventing:

PHASE 3 = READY

For each:
- severity
- component
- problem
- impact
- recommended remediation

=====
NON-BLOCKING ISSUES
=====

List remaining improvements.

=====
STOP CONDITION
=====

Do NOT start Phase 4.

Do NOT add paid services without explicit approval.

Do NOT fabricate data or test results.

Do NOT expose secrets.

Do NOT give users direct GitHub repository write access.

Do NOT delete legitimate cemetery or historical data.

STOP after the final report.

WAIT FOR EXPLICIT INSTRUCTIONS FOR PHASE 4.